

TOO-EASY

DOCUMENTO DE ESTUDIO — MÓDULO 1

Autenticación, Arquitectura y Seguridad del Sistema

Registro

Login

Recuperación

JWT

Seguridad BD

Correos

Preguntas de Repaso

Grupo 6IV7

1 ARQUITECTURA DEL SISTEMA

Modelo utilizado

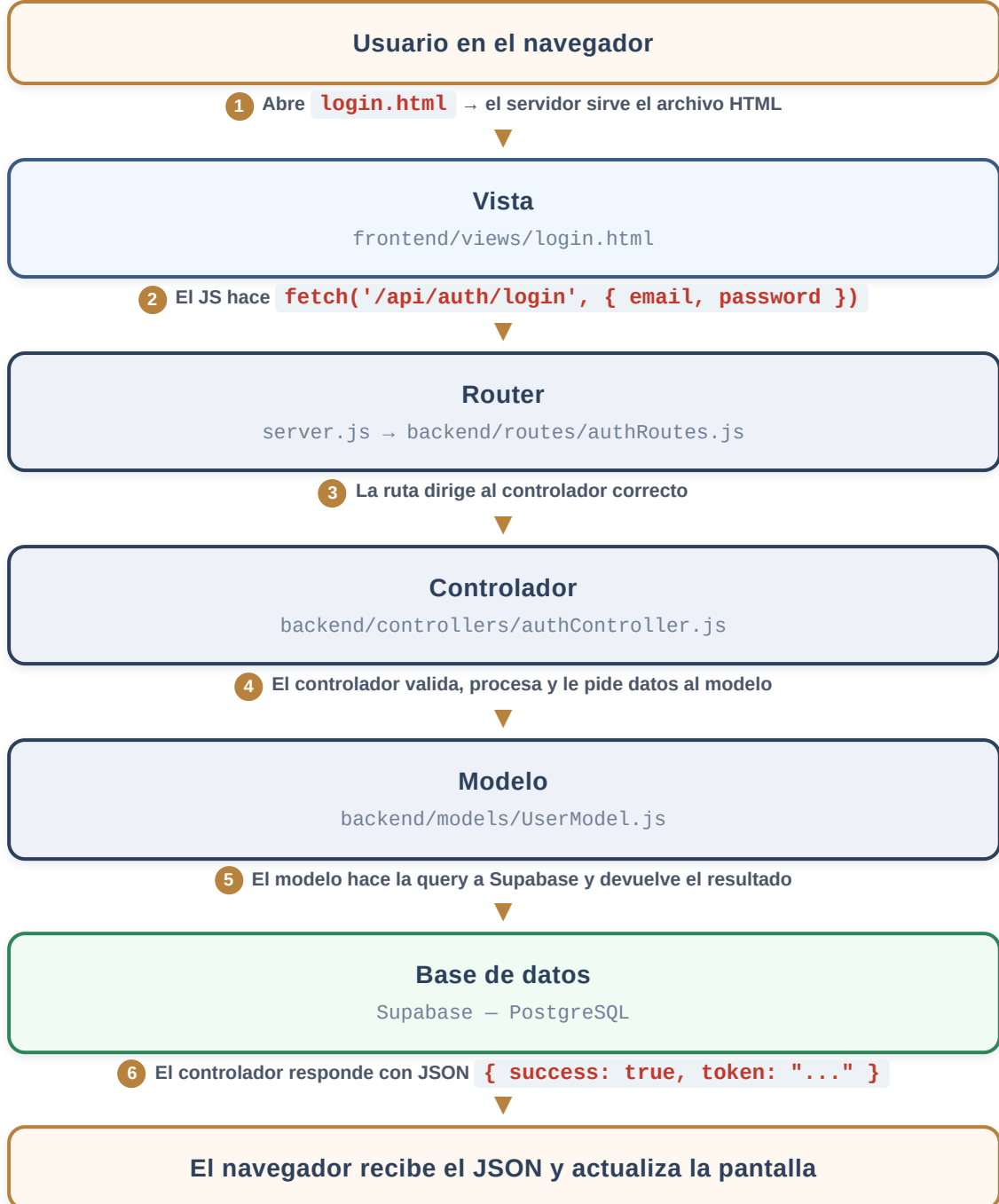
El sistema usa dos modelos combinados:

- **Cliente-Servidor:** hay dos bloques principales — el cliente (lo que ve el usuario en el navegador) y el servidor (todo lo que corre detrás).
- **MVC (Modelo-Vista-Controlador):** patrón que organiza el código del servidor en tres responsabilidades separadas.

¿Qué hace cada capa del MVC?

Capa	Carpeta en el proyecto	Responsabilidad
Vista	<code>frontend/views/</code> + <code>frontend/public/</code>	Lo que el usuario ve: HTML, CSS, imágenes
Controlador	<code>backend/controllers/</code>	Recibe la petición, aplica las reglas, decide qué responder
Modelo	<code>backend/models/</code>	Habla con la base de datos (queries)

Flujo de una petición de principio a fin



Responsabilidades en una línea

FRONT — Lo que el usuario ve y toca. Manda peticiones, recibe JSON, actualiza la pantalla.

BACK — El cerebro. Recibe peticiones, aplica reglas, decide si algo está bien o mal, habla con la BD.

BD — Solo guarda y devuelve datos. No piensa, no valida, no decide.

2 TECNOLOGÍAS USADAS Y POR QUÉ

Tecnología	Para qué	Por qué se eligió
Node.js	Ejecuta JavaScript en el servidor	Permite usar JS tanto en front como en back
Express 5	Framework HTTP	Evita construir el servidor desde cero; estándar de facto
Supabase (PostgreSQL)	Base de datos en la nube	Incluye RLS, backups automáticos y API lista
JWT	Identificar quién está logueado	El servidor no guarda estado; el token viaja en cada petición
bcryptjs	Encriptar contraseñas	Lento a propósito — dificulta ataques de fuerza bruta
Helmet	Cabeceras de seguridad HTTP	Reduce riesgos comunes con una línea de configuración
CORS	Controla qué orígenes acceden al servidor	Evita que sitios externos consuman la API sin permiso
express-rate-limit	Limita peticiones por IP	Previene abuso y ataques de fuerza bruta
Brevo (API HTTP)	Correos transaccionales automáticos	Gratis hasta 300/día, sin configurar servidor de correo
nodemailer + Gmail	Correos masivos manuales (broadcast)	Script que se corre a mano desde la terminal
Cloudflare Turnstile	Anti-bot en registro y login	Captcha inteligente que no interrumpe al usuario normal
dotenv	Lee variables de entorno desde .env	Mantiene credenciales fuera del código fuente
Vanilla JS	Frontend sin frameworks	Sin dependencias; se entiende exactamente qué hace cada línea

Vanilla JS — ventajas y límites

¿Qué es? JavaScript puro sin librerías externas (sin React, Vue, Angular). Ideal para proyectos académicos.

Su límite: cuando la app crece, el código de cada página se vuelve difícil de mantener porque no hay componentes reutilizables ni estado centralizado. Para escalar se recomendaría React o Vue en el front y TypeScript en el back.

¿Qué es un framework?

Conjunto de herramientas y convenciones predefinidas que resuelven problemas comunes para que no tengas que construir todo desde cero. Express maneja el routing HTTP sin que tengas que programar cómo leer una petición TCP raw.

3 ESTRUCTURA DEL PROYECTO

```
T00-EASY/
|
├─ server.js           ← Punto de entrada. Configura todo y levanta el
servidor.
|
├─ backend/
|   ├─ config/supabase.js ← Conexión a la BD (dos clientes: normal y admin)
|   ├─ controllers/      ← Lógica de negocio
|   ├─ models/           ← Queries a la BD
|   ├─ routes/           ← Define qué URLs existen y a qué controlador van
|   ├─ middleware/       ← Filtros que se ejecutan antes del controlador
|   ├─ services/         ← Servicios compartidos
|   └─ utils/            ← Herramientas: email, seguridad, validaciones
|
└─ frontend/
    ├─ views/            ← Páginas HTML
    └─ public/
        ├─ css/          ← Estilos por pantalla
        ├─ js/pages/     ← JS específico de cada página
        ├─ js/modules/   ← JS reutilizable
        └─ img/          ← Imágenes
```

¿Qué es server.js?

Es el punto de arranque. Cuando ejecutas `npm start` lo primero que corre es este archivo. Hace tres cosas: (1) configura middlewares de seguridad, (2) registra todas las rutas, (3) levanta el servidor en el puerto 3000.

¿Qué son los middlewares?

Intermediarios que se ejecutan entre que llega la petición y llega al controlador. Pueden dejar pasar, modificar o rechazar la petición.

Middleware	Qué hace
<code>requireAuth</code>	Verifica que el JWT sea válido antes de dejar pasar
<code>requireLevelAccess</code>	Verifica que el usuario tenga nivel suficiente
<code>helmet()</code>	Agrega cabeceras de seguridad a todas las respuestas
<code>rateLimit()</code>	Bloquea IPs que hagan demasiadas peticiones

4 MÓDULO DE AUTENTICACIÓN — ENDPOINTS

Método	Endpoint	Para qué
POST	<code>/api/auth/register</code>	Crear cuenta nueva
POST	<code>/api/auth/login</code>	Iniciar sesión
POST	<code>/api/auth/verify-email</code>	Verificar código enviado al correo
POST	<code>/api/auth/resend-verification</code>	Pedir otro código de verificación
POST	<code>/api/auth/request-recovery</code>	Solicitar código para recuperar contraseña
POST	<code>/api/auth/verify-recovery-code</code>	Verificar ese código
POST	<code>/api/auth/reset-password</code>	Guardar nueva contraseña

¿Por qué todo POST? Los datos sensibles (contraseña, código) no deben ir en la URL. POST manda los datos en el cuerpo de la petición, que viaja encriptado con HTTPS.

5 FLUJO DE REGISTRO

■ FRONT ■ BACK ■ BD

- 1 FRONT Muestra el formulario de registro
- 2 FRONT Envía datos con `fetch POST` → el back los recibe
- 3 BACK Verifica Cloudflare Turnstile (anti-bot)
- 4 BACK Valida campos: nombre, email, contraseña, edad
- 5 BD Consulta si email/nombre ya existen → busca en tabla `users`
- 6 BACK Verifica contraseña contra HIBP (consulta API externa)
- 7 BACK Encripta la contraseña con `bcrypt`
- 8 BD Crea el usuario con `email_verified = false` → INSERT en `users`
- 9 BD Genera código 6 dígitos, lo hashea (SHA-256) → INSERT en `verification_codes`
- 10 BACK Manda el código al correo (Brevo)
- 11 FRONT Muestra pantalla para ingresar el código `{ requiresVerification: true }`
- 12 BD Usuario ingresa código → back verifica hash → actualiza `email_verified = true`
- 13 BACK Genera y devuelve el JWT → el front lo guarda y redirige al juego

Validaciones del registro

- Nombre: entre 3 y 16 caracteres
- Contraseña: entre 8 y 64 caracteres
- Email: formato válido (regex)
- Edad: número entero entre 15 y 99
- Email/nombre: no deben existir ya en la BD
- Contraseña: no debe estar en bases de datos de filtraciones conocidas (HIBP)

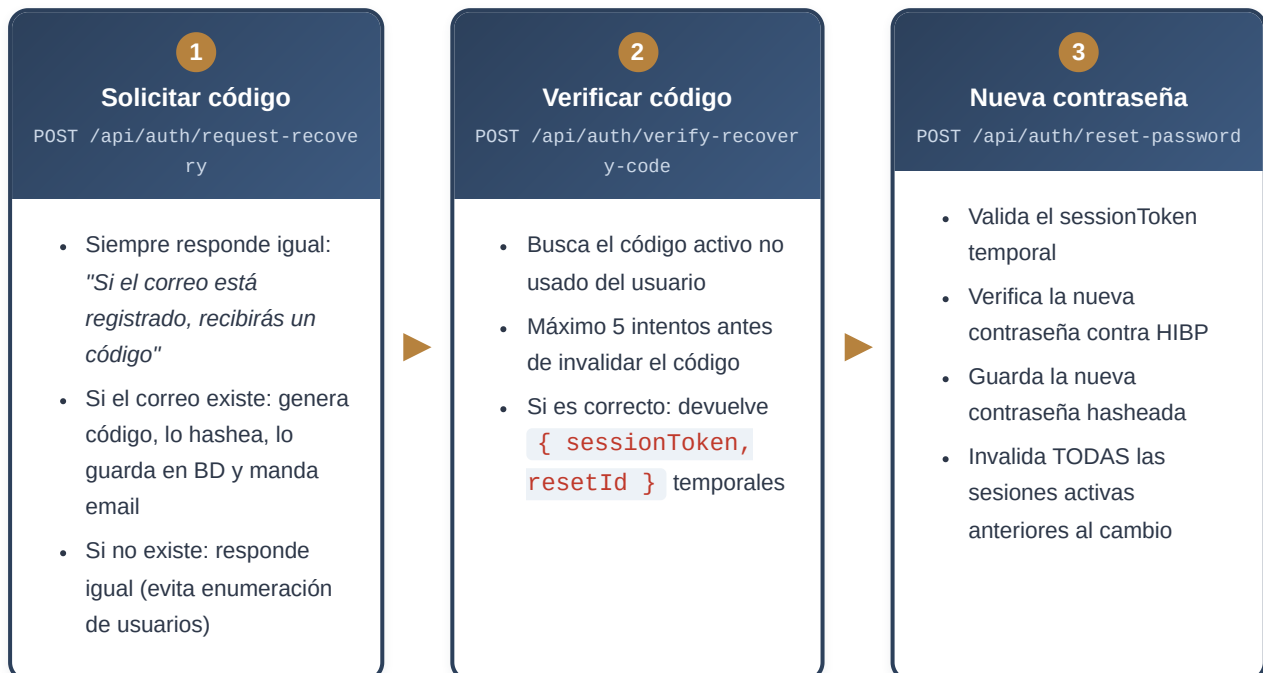
¿Qué pasa si el usuario no verifica su correo? La cuenta queda con `email_verified = false`. No puede hacer login. Si pasan 24 horas sin verificar, una tarea automática la elimina de la BD (la persona puede volver a registrarse con el mismo email y nombre).

6 FLUJO DE INICIO DE SESIÓN

■ FRONT ■ BACK ■ BD

- 1 **FRONT** Muestra formulario de inicio de sesión
- 2 **FRONT** Envía email + contraseña → el back los recibe
- 3 **BACK** Verifica Cloudflare Turnstile
- 4 **BD** Busca usuario por email → SELECT en `users`
- 5 **BACK** Si no existe: bcrypt dummy (protección timing attack)
- 6 **BACK** Verifica si la cuenta está bloqueada (`locked_until`)
- 7 **BACK** Compara contraseña con hash guardado (bcrypt)
- 8 **BD** Si falla: suma intentos; al llegar a 5 bloquea 15 min → UPDATE `failed_login_attempts`
- 9 **BACK** Verifica que `email_verified = true`
- 10 **BD** Login exitoso: genera JWT, guarda IP/fecha → UPDATE `last_login_at` , resetea intentos
- 11 **FRONT** Guarda JWT y redirige al juego (recibe JWT + datos del usuario)

7 FLUJO DE OLVIDE MI CONTRASEÑA



8 EL TOKEN JWT — ¿QUÉ ES Y CÓMO FUNCIONA?

JWT (JSON Web Token) es una cadena de texto firmada digitalmente que contiene información del usuario. El servidor la genera al hacer login y el cliente la guarda.

¿Por qué JWT y no sesiones tradicionales?

Con sesiones tradicionales el servidor guarda en memoria "el usuario X está logueado". Con JWT el servidor no guarda nada — el token viaja en cada petición y el servidor solo verifica la firma. Ventaja: el servidor puede reiniciarse sin perder sesiones.

¿Qué contiene el token en este proyecto?

```
Token = base64url( userId . issuedAt . firmaHMAC )
```

¿Cómo se usa en cada petición?

El frontend manda:

Header: Authorization: Bearer <token>

El middleware requireAuth:

1. Lee el token del header
2. Verifica la firma HMAC
3. Verifica que no haya expirado (7 días)
4. Verifica que no sea anterior al último cambio de contraseña
5. Mete req.userId para que el controlador lo use

Invalidación automática: si el usuario cambia su contraseña, todas las sesiones anteriores quedan invalidadas. Esto se logra guardando `password_changed_at` en la BD y comparándolo con `issuedAt` del token.

9 MEDIDAS DE SEGURIDAD EN EL BACKEND

bcrypt — encriptación de contraseñas

Algoritmo de hash diseñado para ser lento. Con 12 rounds tarda ~250ms por contraseña, haciendo un ataque de fuerza bruta imprácticamente lento. ¿Por qué no MD5 o SHA-256? Porque son rápidos — un atacante puede probar millones de combinaciones por segundo. bcrypt es intencionalmente lento.

HIBP — Have I Been Pwned

Base de datos pública con contraseñas robadas en brechas conocidas. Se consulta con **k-anonymity**: se mandan solo los primeros 5 caracteres del hash SHA-1 de la contraseña. La API devuelve los hashes que empiecen así y el sistema verifica localmente. La contraseña nunca sale completa del servidor.

Cloudflare Turnstile — anti-bot

Captcha inteligente que analiza el comportamiento del usuario. Si detecta algo sospechoso muestra un desafío visual; si no, lo deja pasar sin interrumpirlo. Se usa en registro y login.

Timing Attack — protección

Si el servidor responde más rápido cuando el usuario no existe, un atacante puede medir esos tiempos para saber qué correos están registrados. **Solución**: cuando el usuario no existe, el backend hace bcrypt con un hash dummy de todas formas, nivelando el tiempo de respuesta.

Rate Limiting — límites de peticiones

Límite	Valor
Peticiones generales	300 / 15 min por IP
Endpoints de auth	20 / 15 min por IP
Solicitudes de código	3 / 15 min por usuario
Intentos de código	5 intentos → código invalidado
Intentos de login fallidos	5 intentos → cuenta bloqueada 15 min

Helmet — cabeceras de seguridad

Agrega automáticamente cabeceras HTTP que protegen contra ataques comunes: CSP (controla qué recursos carga la página), X-Frame-Options: DENY (evita clickjacking), HSTS (fuerza HTTPS), X-Content-Type-Options: noSniff.

10 SEGURIDAD EN LA BASE DE DATOS (SUPABASE)

Lo que Supabase da automáticamente

Qué	Para qué
SSL/TLS	Toda conexión entre backend y BD viaja encriptada
Backups automáticos	Supabase guarda copias periódicamente
API Keys	Solo quien tenga las keys puede hablar con la BD
RLS (Row Level Security)	Políticas que controlan quién puede ver qué filas

Los dos clientes de Supabase

Cliente	Key usada	Qué puede
<code>supabase</code>	ANON KEY	Respetar políticas RLS — para lecturas
<code>supabaseAdmin</code>	SERVICE ROLE KEY	Bypasea RLS — para escrituras y operaciones críticas

Campos de seguridad en la tabla `users`

Campo	Para qué
<code>password_hash</code>	Contraseña nunca en texto plano
<code>email_verified</code>	No se puede entrar sin verificar el correo
<code>is_active</code>	Cuentas desactivadas no pueden hacer login
<code>failed_login_attempts</code>	Contador de intentos fallidos
<code>locked_until</code>	Fecha hasta la que está bloqueada la cuenta
<code>password_changed_at</code>	Para invalidar sesiones anteriores al cambio
<code>last_login_ip</code>	Detectar accesos desde nueva IP
<code>last_login_ua</code>	Dispositivo/navegador del último acceso

Índice en `password_changed_at`

Cada request autenticado hace una query para verificar si la sesión sigue válida. Sin índice PostgreSQL escanea toda la tabla en cada petición. Con índice la búsqueda es instantánea. Los índices son estructuras auxiliares como el índice de un libro.

11 SOFT DELETE vs HARD DELETE

Soft Delete: marcar con `is_active = false` en vez de borrar físicamente. El registro sigue en la BD pero el sistema lo trata como si no existiera. Permite reactivar la cuenta.

Hard Delete: borrado físico con `DELETE FROM tabla WHERE id = X`. No hay vuelta atrás.

Quién borra	Cómo	Reversible	Tiempo
Admin desactiva cuenta	<code>is_active = false</code> (soft delete)	Sí	Hasta 90 días
Usuario borra su cuenta	DELETE directo (hard delete)	No	Inmediato
Cuenta zombie (no verificó)	DELETE automático	No	A las 24 horas
Cuenta inactiva 90 días	DELETE automático	No	A los 90 días

El trigger eliminado: existía uno llamado `trg_hard_delete_on_deactivate` que interceptaba cualquier `UPDATE SET is_active = false` y lo convertía en un DELETE. Se eliminó porque causaba un bug y porque la lógica se movió al backend para tener más control.

12 SISTEMA DE CORREOS

Sistema	Tecnología	Para qué	Cómo se activa
<code>emailService.js</code>	Brevo (API HTTP)	Correos automáticos del sistema	Se dispara solo cuando el usuario hace algo
<code>send-broadcast.js</code>	Gmail + nodemailer (SMTP)	Correos masivos manuales	A mano desde la terminal

Brevo vs SMTP

SMTP conecta directamente a un servidor de correo por puerto 587/465. Brevo es un intermediario: el backend manda un `fetch` HTTPS con la API key y Brevo se encarga de entregar el correo. Más fácil, mejor tasa de entrega, gratis hasta 300/día.

¿Cómo se genera y verifica el código de 6 dígitos?

- `crypto.randomInt(100000, 999999)` → número aleatorio sin sesgo
- `hashCode(code)` → SHA-256 del código
- Guarda el HASH en `verification_codes` (nunca el código en texto plano)
- Manda el código original al correo del usuario
- Usuario ingresa el código → se hashea → se compara con el hash guardado

¿Por qué los correos usan estilos en línea?

Los clientes de correo (Gmail, Outlook) bloquean hojas de estilo externas. El único CSS que respetan es el pegado directamente en cada elemento con `style="..."`. Por eso todos los correos tienen `style="color:#2C405B"` directo en cada etiqueta.

¿Qué hace el paquete `resend` ? Está instalado en `package.json` pero **nunca se usa**. Es una dependencia muerta de cuando el proyecto usaba Resend antes de migrar a Brevo. La función `resendVerificationCode` en el código no tiene nada que ver con el servicio Resend — solo significa "volver a enviar el código".

13 LIMPIEZAS AUTOMÁTICAS (CRON JOBS)

Tarea	Condición	Frecuencia
Tickets expirados	Tickets con más de 30 días	Cada 24 horas
Cuentas zombie	<code>email_verified = false</code> + más de 24 horas desde el registro	Cada 24 horas
Cuentas inactivas	<code>is_active = false</code> + más de 90 días sin reactivarse	Cada 24 horas

Todas se ejecutan también unos segundos después de que el servidor arranca para limpiar lo acumulado.

Algoritmo

Secuencia de pasos para resolver un problema. Es una receta, no código. Ej: *bcrypt* define cómo hashear una contraseña.

Función

Bloque de código reutilizable que ejecuta una tarea concreta. Ej: `hashPassword(plainText)`

Método

Una función que pertenece a un objeto. Ej: `supabase.from('users').select()` — *select* es un método del objeto *supabase*.

Lógica de negocio

Las reglas que definen qué debe pasar y cuándo. Ej: "si hay 5 intentos fallidos, bloquear 15 minutos".

Endpoint

URL específica de la API que acepta peticiones. Ej: `POST /api/auth/login`

Hash

Transformación de texto a una cadena de longitud fija que no se puede revertir. Ej: "hola123" → "\$2b\$12\$..."

Salt

Valor aleatorio añadido a la contraseña antes de hashearla para que dos contraseñas iguales produzcan hashes diferentes.

JWT (JSON Web Token)

Token firmado que contiene información del usuario. El servidor lo genera al login y el cliente lo manda en cada petición para identificarse.

Middleware

Función que se ejecuta entre que llega la petición y llega al controlador. Puede dejar pasar, modificar o rechazar la petición.

Trigger (BD)

Acción automática que la BD ejecuta sola cuando ocurre un evento (INSERT, UPDATE, DELETE), sin que el backend se lo pida.

Soft Delete

Marcar un registro como inactivo (`is_active = false`) en vez de borrarlo físicamente. Permite reactivarlo después.

Hard Delete

Borrado físico de una fila. `DELETE FROM tabla WHERE id = X` . No hay vuelta atrás.

Índice (BD)

Estructura auxiliar que hace las búsquedas más rápidas. Como el índice de un libro — en vez de leer todo, vas directo a la página.

RLS (Row Level Security)

Sistema de PostgreSQL que define políticas sobre qué filas puede ver o modificar cada tipo de conexión.

SQL Injection

Ataque donde se mete código SQL en un campo de texto para manipular las queries. Se previene con queries parametrizadas y validación de inputs.

Timing Attack

Ataque donde se miden tiempos de respuesta para deducir información. Se previene nivelando los tiempos de respuesta con operaciones dummy.

K-Anonymity

Técnica para consultar una API sin revelar el dato completo. En HIBP: se mandan los primeros 5 caracteres del hash, no la contraseña completa.

Rate Limiting

Límite de peticiones por IP en un período de tiempo. Previene abuso y ataques de fuerza bruta.

Brute Force

Ataque que prueba millones de combinaciones de contraseñas hasta encontrar la correcta. Se mitiga con bcrypt, rate limiting y bloqueo de cuenta.

Framework

Conjunto de herramientas predefinidas que resuelven problemas comunes para no construir todo desde cero. Express es un framework HTTP para Node.js.

¿Por qué se hashea la contraseña antes de guardarla?

Porque si alguien roba la BD no obtiene las contraseñas reales. Solo obtiene hashes que no se pueden revertir directamente. El usuario puede seguir usando esa contraseña en otros servicios sin que el atacante la sepa.

¿Por qué bcrypt y no MD5 o SHA-256 para contraseñas?

MD5 y SHA-256 son rápidos — un atacante puede probar millones de combinaciones por segundo. bcrypt es intencionalmente lento (~250ms por hash con 12 rounds), haciendo los ataques de fuerza bruta imprácticamente lentos.

¿Qué es un timing attack y cómo se previene?

Si el servidor responde más rápido cuando el usuario no existe, un atacante puede medir esos tiempos para saber qué correos están registrados. Se previene haciendo bcrypt con un hash dummy cuando el usuario no existe, nivelando el tiempo de respuesta en ambos casos.

¿Qué pasa si alguien intenta registrarse con un correo que ya existe?

El backend consulta la BD antes del INSERT. Si encuentra el correo o el nombre, responde con error 409 (Conflict) indicando qué campo está duplicado, sin crear nada.

¿Por qué la recuperación siempre dice "si el correo está registrado te llegará un código"?

Para evitar enumeración de usuarios. Si dijera "ese correo no existe", un atacante podría probar miles de correos y armar una lista de cuáles están registrados en el sistema.

¿Qué es el soft delete y por qué se usa?

Es desactivar con `is_active = false` en vez de borrar físicamente. Se usa para conservar historial, poder reactivar cuentas y mantener integridad referencial — si el usuario tiene movimientos o lecciones, borrar la fila rompería esas relaciones.

¿Qué es RLS y para qué sirve en Supabase?

Row Level Security define políticas sobre qué filas puede ver cada tipo de conexión. Funciona como segunda capa de seguridad — aunque alguien acceda a Supabase directamente, solo ve lo que las políticas permiten.

¿Por qué hay dos clientes de Supabase?

El cliente anon respeta las políticas RLS y se usa para lecturas. El cliente admin bypasea RLS y se usa para escrituras y operaciones críticas. Principio de mínimo privilegio: cada operación usa solo el acceso que necesita.

¿Qué es un índice en la BD y por qué crearon uno en `password_changed_at` ?

Un índice hace búsquedas más rápidas, como el índice de un libro. Se creó ahí porque cada request autenticado hace una query para verificar si la sesión sigue válida. Sin índice PostgreSQL recorre toda la tabla en cada petición; con índice es instantáneo.

¿Por qué el código de verificación se guarda hasheado?

Si alguien accede a la BD no obtiene los códigos activos y no puede usarlos para verificar cuentas ajenas. El código real solo existe en el correo del usuario — en la BD solo vive su huella (hash SHA-256).

¿Qué pasa si el usuario se registra pero nunca verifica su correo?

La cuenta queda con `email_verified = false` y no puede hacer login. Si pasan 24 horas sin verificar, una tarea automática del servidor la elimina. La persona puede volver a registrarse con el mismo correo y nombre.

¿Por qué la sesión se invalida cuando el usuario cambia su contraseña?

Si alguien robó el token de sesión de otro usuario, al cambiar la contraseña todas las sesiones anteriores quedan inútiles. Esto se logra guardando `password_changed_at` y comparándolo con `issuedAt` del JWT en cada request.

¿Qué es HIBP y cómo funciona sin enviar la contraseña completa?

Have I Been Pwned es una BD de contraseñas filtradas en brechas conocidas. Se consulta con k-anonymity: se mandan solo los primeros 5 caracteres del hash SHA-1. La API devuelve hashes que empiecen así y el sistema verifica localmente. La contraseña nunca sale completa del servidor.

¿Qué diferencia hay entre cómo borra su cuenta el usuario vs el admin?

El usuario hace un DELETE directo e inmediato (hard delete, sin recuperación). El admin pone `is_active = false` (soft delete, recuperable hasta 90 días). Si el admin no reactiva en 90 días, una tarea automática lo elimina definitivamente.

Diferencia entre algoritmo, función, método y lógica

Algoritmo: la receta (secuencia de pasos para resolver algo). **Función:** el código que ejecuta esa receta. **Método:** una función que pertenece a un objeto. **Lógica:** las reglas de negocio que dicen qué debe pasar y cuándo.